

Intelligent Financial Fraud Detection — Technical Proposal

NTU Datathon 2026 · Track 2 (Finance) · Team **The Larpers** · all metrics are out-of-fold (3× repeated stratified 5-fold CV)

1. Problem understanding

Binary classification of 20,000 labelled transactions with only **1.76% fraud** (353 cases). Accuracy is meaningless (always "legit" = 98%). We split the problem in two: **(i)** build the best possible **risk score**, judged by PR-AUC (random = 0.018); **(ii)** choose a **cost-aware decision threshold** that balances frauds caught (recall) against disruption to good customers (precision), summarised by F1.

2. Data exploration — key insights

- **Clean data:** no duplicates or train/test overlap; id is a sequential counter with a flat fraud rate → dropped (no leakage). ~0.6% missing per field.
- **Fraud signatures:** young account (<90 days) + ≥4 transactions in the last hour = **40% fraud**; new device 7.5% (17% if also foreign); >10 transactions/24h 13–28%; \$1.5–4k cash transfers / electronics 11–14%.
- **Non-linear amount:** tickets above \$4k are *less* risky than \$1.5–4k (0 frauds above \$4k in cash transfers or electronics), so "bigger = riskier" is not safe.
- **Noisy labels:** even the riskiest segments are <50% fraud, so a realistic ceiling is PR-AUC ≈ 0.2 and F1 ≈ 0.3.
- **Population shift:** adversarial validation separates train from test (AUC 0.67). The test set has 3× more missing values, 3× more high-velocity transactions and a heavier amount tail; train rows that look like test have 2× the fraud rate → we design for **robustness**, not for the leaderboard.

3. Features & validation

A single defensive `make_features()` is used everywhere. It coerces `text/±inf/negatives`, normalises and tolerates unseen categories, and creates absent columns. On top of the 10 raw inputs it adds 7 interpretable features: amount vs the customer's average ticket, amount vs the category median, burst ratio (1h/24h activity), night, foreign, young account (<180 days) and number of missing fields. We tested 17 engineered features; with trees, more added no measurable lift. **Validation:** repeated stratified 5-fold CV. The PR-AUC noise is ±0.006, so smaller gaps are ignored. Our 40-configuration search showed a *winner's curse*: the best config fell from 0.215 to 0.206 when re-validated. The test set is never used for model decisions.

4. Models compared (same folds)

Model	PR-AUC	ROC-AUC
Logistic regression (baseline)	0.194	0.769
Random forest	0.188	0.766
Gradient boosting – default	0.165	0.720
Gradient boosting – tuned	0.208	0.762
LightGBM / XGBoost – tuned	0.209 / 0.200	0.763 / 0.768
FINAL (3 CV repeats: 0.206 ± 0.003)	0.208	0.771

`class_weight="balanced"` consistently *hurt* PR-AUC (–0.01 to –0.04). We handle the imbalance at the threshold instead.

5. Final model and why

5 seed-bagged gradient-boosting models (scikit-learn `HistGradientBoostingClassifier`: depth 4, 8 leaves, learning rate 0.01, 800 trees, ≥80 samples per leaf, 50% feature subsampling), averaged.

- **Gradient boosting:** captures the interactions found in the EDA (young account × burst, new device × foreign); handles missing values natively; insensitive to scaling and outliers; best PR-AUC.
- **Monotonic constraints:** new device ↑, velocity ↑, spend ↑, account age ↓, with amount left free. This is domain knowledge as a guarantee for the extreme, under-represented cases that are 3× more common in test. It costs about 0.004 PR-AUC (within noise).
- **Bagging:** lowers the variance of a model trained on only 353 frauds (±0.003 across CV repeats).
- **Missing-value augmentation:** training also uses a copy with 3% of each field blanked. This raised PR-AUC from 0.205 to 0.214 on clean validation and from 0.198 to 0.207 on gappy validation.

6. Decision threshold

We maximise **F1.5** (recall weighted 1.5× precision) on the averaged out-of-fold curve, which gives **t = 0.095**. In business terms: act when $P(\text{fraud}) > 9.5\%$, i.e. a missed fraud is treated as ~10× as costly as a false alarm (one extra OTP check). This choice is stable across CV repeats (0.068–0.098), unlike the F1-optimum (0.12–0.30, a flat curve). The threshold is stored inside `model.pkl` (`FixedThresholdClassifier`): `predict()` returns decisions and `predict_proba()` returns scores.

7. Results (out-of-fold)

PR-AUC 0.206 ± 0.003 (≈12× random) · ROC-AUC 0.77. At $t = 0.095$ the model flags **1.7%** of transactions with **precision 0.27, recall 0.27, F1 0.27**: it catches 27% of frauds and **46% of fraud dollars**. One alert in 3.7 is real fraud, and only 1.3% of good customers are challenged. The most important inputs (out-of-fold permutation importance) are **new device, account age, velocity (1h + 24h) and amount**. Every response curve matches the EDA. On the shifted test set the model flags 2.7%, consistent with a riskier population.

8. Assumptions, limitations, improvements

- **Assumptions:** transactions are independent (no customer ID or timestamp, so stratified rather than temporal CV); labels are correct; the 1h/24h aggregates are computed before the transaction (no leakage); fraud mechanisms are stable while the customer mix shifts.
- **Limitations:** noisy labels cap precision, so the model should rank a review / step-up queue rather than auto-block. Regions above \$4k or 7+ transactions/hour are thin in training data (the constraints make them safe, not certain). The threshold assumes a 10:1 cost ratio.
- **Robustness tested:** the saved model scores rows with text, `±inf`, negatives, unseen categories and missing columns without errors.
- **Next steps:** amount-aware decisions (flag when $P \times \text{amount} > \text{review cost}$); customer IDs and timestamps for per-customer baselines and temporal validation; device and IP signals; drift monitoring with monthly threshold recalibration.